CloudSage — Project Description

AI-powered AWS infrastructure cost optimization platform

1. Executive Summary

CloudSage is a DevOps + AI platform that discovers AWS infrastructure, combines cost data with real utilization, health, configuration and Terraform context, identifies optimization candidates, asks one selected LLM to reason over the evidence, produces explainable recommendations, and eventually creates safe Terraform changes that require validation and human approval.

The production system will use ONE LLM API key/provider at runtime. Multiple AI systems may be used by the development team while building the project, but they are development assistants only and are not part of the deployed product.

2. Purpose

The purpose is to solve the gap between AWS billing/monitoring data and an actionable infrastructure decision: what should be changed, why, how much could it save, what risk exists, how to implement the change through IaC, and whether the optimization actually worked.

3. Problem Being Solved

- AWS cost data, CloudWatch metrics, infrastructure configuration and Terraform code are fragmented.
- Human engineers must correlate cost, utilization, configuration and operational history manually.
- Simple rules can find obvious waste but cannot reliably understand context and operational risk.
- LLMs can reason about context, but they must not be trusted with unrestricted infrastructure write access.
- Optimization recommendations need evidence, deterministic financial calculations, Terraform validation, approval, auditability and post-change verification.

4. What CloudSage Is Not

- Not an AWS Cost Explorer clone.
- Not a generic chatbot connected directly to AWS.
- Not an autonomous AI with administrator credentials.
- Not an LLM that performs financial calculations itself.
- Not a multi-agent runtime. Runtime starts with one selected LLM API.

5. Core Value

CloudSage turns fragmented infrastructure evidence into a controlled optimization lifecycle: Discover → Measure → Detect → Reason → Recommend → Approve → Plan → Apply → Verify → Record.

6. Initial Scope

The first implementation supports one AWS service/resource family: EC2. It collects EC2 metadata, Cost Explorer data and CloudWatch utilization, identifies rightsizing candidates, sends structured context to the LLM, and returns a structured recommendation. Terraform integration and apply automation are added only after the EC2 analysis path is stable.

7. Long-Term Scope

- EC2 rightsizing
- EBS cleanup/optimization
- Elastic IP waste detection
- S3 storage/lifecycle optimization
- RDS rightsizing
- ALB optimization
- ECS optimization
- Lambda optimization
- NAT Gateway and data-transfer analysis
- Cross-resource and dependency-aware recommendations
- Savings verification against actual billing

8. Success Criteria

- Every recommendation has evidence.
- Financial calculations are performed deterministically.
- AI outputs are structured and machine-readable.
- Production changes require explicit approval.
- Terraform is the preferred infrastructure change mechanism.
- Unexpected Terraform plan impact blocks execution.
- Post-change metrics and cost are compared with the baseline.
- Every recommendation and action is auditable.